Tensile Workbench External Audit Request

Please conduct an independent audit of **Tensile Workbench**, a Python application for analysing tensile-test data, comparing specimen groups, and producing publication plots and Excel exports.

Much of this application has evolved through repeated AI-assisted development. I want an evidence-based assessment of its scientific correctness, reliability, maintainability, and protection of user settings. Prioritise problems that could silently produce incorrect research results or lose saved work.

This is an **audit-only assignment**. Report findings and recommendations; wait for approval before changing application code.

Start by reading `AGENTS.md`, `README.md`, the implementation plan, relevant technical documentation, and recent Git history. Record the exact commit audited. Treat documentation, comments, existing tests, and previous claims of successful verification as claims to check independently.

Protect the installation and research data

- `data` and `output` may be symlinks to real research folders. Do not modify them or regenerate research outputs.
- Do not modify personal project files, folder settings, saved graphs, overrides, or existing backups.
- Use bundled synthetic examples and temporary fixtures. Isolate test outputs, caches, and backups; in particular, point `TENSILE_WORKBENCH_BACKUP_DIR` to a temporary directory.
- Do not upload private data or settings, install or upgrade dependencies without agreement, commit, or push.
- If a finding needs real data to reproduce, identify the smallest anonymised example required.

Please investigate the following areas.

1. Scientific calculations and data transformations

Trace raw CSV import through cleaning, fitting, fracture detection, reconstruction, aggregation, plotting, and export.

Check units, strain conventions, stress conversion, elastic fitting, offset yield strength, UTS, uniform elongation, failure elongation, toughness, and work-hardening calculations. Examine interpolation, smoothing, differentiation, sorting, duplicate values, missing values, and integration boundaries.

Pay particular attention to:

- Brittle and ductile fracture detection, gradual unloading, truncated records, acquisition gaps, noisy readings, and manual endpoint overrides.
- Whether gap handling avoids both false fracture detections and unnecessarily abandoning valid later data.
- Gauge-length reconstruction assumptions, behaviour before and after peak load, and targets above or below the measured gauge length.
- Whether reordering or trimming preserves corresponding stress, strain, force, time, and source-row relationships.
- Whether display rounding ever affects calculations, identity checks, saved overrides, or exported numeric precision.

Verify any standards-related claims against authoritative sources where available. Identify the edition and clause relied upon. Clearly distinguish implemented heuristics from verified standards requirements.

2. Specimen identity, exclusions, and statistical populations

The intended distinctions are important:

- The operator-entered specimen label identifies the physical specimen.
- A CSV filename’s acquisition-row suffix is not necessarily the physical specimen number.
- Identity checking uses name/row information and compatible UTS evidence. Failure elongation must not be used to establish identity.
- Ambiguous names, summary revisions, duplicate labels, and mismatched data should be surfaced rather than silently resolved incorrectly.

Audit global exclusions, graph-specific overrides, partial AVE failures, and shape-only exclusions. Eligible strength values should remain available when elongation or curve shape is excluded.

For landmark averaging, check that the resulting curve passes through the intended reported group means while its shape uses only eligible contributors. Verify contributor counts, error bars, and population labels. Check the strength-elongation scatter options for independently eligible versus paired populations.

Investigate whether tables, inspectors, plots, and exports consistently apply these rules.

3. Saved state, migrations, backups, and recovery

Review atomic writes, locking, concurrent sessions, conflict detection, schema migrations, undo, graph duplication/deletion, and recovery from interrupted or corrupt saves.

Check that:

- Upgrades preserve personal settings and graph definitions.
- Generic defaults remain separate from personal state and machine-specific paths.
- Folder moves migrate the correct identities and settings only after approval.
- Ambiguous moves and duplicate fingerprints cannot silently transfer overrides to another specimen.
- Backup integrity checks, retention, protected snapshots, and restore operations behave as documented.
- Failed backup or restore operations give accurate feedback and preserve recoverable data.
- Startup and reload do not silently overwrite valid settings with stale or default values.

Include cross-platform path handling and installations in cloud-synchronised folders.

4. Custom-variable editors and UI state

The latest changes made both variable editors table-only. Review the complete draft → validation → save → reload lifecycle.

Verify:

- Row 1 contains names, row 2 units, and subsequent rows values.
- Rectangular CSV/TSV paste, quoting, blank cells, added columns, copying, and invalid input behave predictably.
- Changing group selection/order, showing all rows, switching graphs, or refreshing does not lose unsaved edits or overwrite newer saved values.
- Deletion affects the intended variable and handles dependent plots coherently.
- Required variables cannot be deleted or renamed incorrectly.
- Specimen count displays read-only “X tested of” beside editable total Y. X includes tested specimens regardless of analysis exclusion; Y is a whole number and cannot fall below the tested count.
- Location defaults come from physical specimen labels across supported naming formats, accept integer overrides, and restore the default when an override is cleared.
- Untested positions can appear on specimen charts and trendline domains without becoming fabricated observations.

Assess error feedback and loading behaviour as well as underlying data correctness.

5. Plots, exports, and performance

Check consistency between static plots, interactive plots, tables, and Excel exports, including units, provenance, population counts, custom variables, overrides, and exclusions.

Verify that labels distinguish Instron, Calc, and Reconstruct appropriately. Confirm that cosmetic labels do not replace stable identities.

Investigate unnecessary repeated calculation, cache invalidation, large-dataset behaviour, “show all” actions, and exporting plots with tables. Look for stale cached results after changing data or settings.

Review specimen-location trendlines, missing observations, extrapolation, and sample-group ordering.

6. Tests, deployment, and maintainability

Run the existing tests in isolation where possible. Independently assess whether they test meaningful behaviour and catch regressions. Look for tests that simply repeat implementation logic, expectations weakened to accept a bug, mocks hiding integration failures, and important untested paths.

Use temporary adversarial examples or independent numerical calculations to investigate suspicious behaviour. Do not change expected results merely to make tests pass.

Review macOS/Windows setup and launch paths, dependency assumptions, local-server exposure, handling of untrusted filenames/labels, and export safety. Identify architectural duplication or coupling where it creates concrete correctness or maintenance risks.

Requested report

1. A short assessment of the application’s reliability and the most consequential risks.
2. Findings ordered by severity, with:
    - File/function and precise line references.
    - Trigger and expected versus actual behaviour.
    - Reproduction steps or a minimal example.
    - Impact on results, saved state, or usability.
    - A proportionate recommended fix.
3. Clearly separate confirmed defects, suspected problems requiring further evidence, and optional improvements.
4. List checks actually performed, their results, and areas you could not verify. Include relevant test commands and failures.
5. Finish with a prioritised remediation plan and the regression checks each fix should add.

Prefer a few well-supported findings over a long speculative list. Do not infer correctness from a green test suite or claim a standards requirement you could not verify.